AI-Powered Blockchain Security

Ethereum Fraud Detection using Machine Learning

Capstone Project Technical Documentation Report

Prepared by:
Kareem Ashraf Talaat Sayed
Ahmed Ahmed Abdel-Salam Ahmed
Ali ElDin Tarek Ali Ibrahim
Ahmed Fouad Mady Mohamed
Mostafa Mahmoud Ahmed Mohamed

Faculty of Engineering, Capital University

Academic Year 2026

Contents

1 Abstract **1**

2 Introduction **1**

2.1 Ethereum and the Blockchain Ecosystem 1

2.2 The Problem of Blockchain Fraud 2

2.3 Role of Machine Learning 2

3 Problem Definition **3**

4 Dataset Description **3**

4.1 Feature Categories 4

5 Preprocessing Pipeline **4**

6 Machine Learning Models **6**

6.1 XGBoost — eXtreme Gradient Boosting 6

6.2 LightGBM — Light Gradient Boosting Machine 6

6.3 Random Forest 6

6.4 Logistic Regression 7

6.5 K-Nearest Neighbours (KNN) 7

6.6 TensorFlow Deep Neural Network (DNN) 7

7 Safety Ensemble Strategy **8**

8 Model Comparison and Analysis **8**

9 Deployment: Streamlit Application **9**

10 Conclusion **11**

11 Data and Code Availability **12**

List of Figures

1 Interface showing Ethereum transaction samples used for analysis. 3
2 Class distribution of fraudulent vs legitimate accounts in the dataset. . . 4
3 Model performance visualization from the training environment. 9
4 Live detection showing the Safety Ensemble flagging a fraudulent transaction. 10
5 Multi-model consensus identifying a safe transaction. 10

List of Tables

1 Model Performance Comparison Summary. 8

1 Abstract

Fraudulent activity on the Ethereum blockchain represents a significant and growing threat to the integrity of decentralised finance. This report presents a comprehensive machine learning system designed to classify Ethereum wallet addresses as either fraudulent or legitimate based on on-chain transactional behaviour. The system leverages a curated dataset of 9,841 labelled Ethereum account records, encompassing 47 engineered behavioural and statistical features derived from native ETH transfers and ERC-20 token movements.

Six distinct machine learning architectures were trained, evaluated, and compared: XGBoost, LightGBM, Random Forest, Logistic Regression, K-Nearest Neighbours (KNN), and a TensorFlow Deep Neural Network (DNN). A Safety Ensemble was additionally constructed by aggregating model votes with a majority-rule scheme tuned for high fraud sensitivity. The best-performing individual model, XGBoost, achieved 99.49% accuracy with a fraud recall of 97.71%. The Safety Ensemble achieved 99.58% accuracy and 98.50% fraud recall, making it the most suitable configuration for production deployment. The system is deployed as an interactive Streamlit web application that enables real-time per-transaction fraud scoring and multi-model comparison.

2 Introduction

2.1 Ethereum and the Blockchain Ecosystem

Ethereum is an open-source, decentralised blockchain platform launched in 2015 by Vitalik Buterin. Unlike Bitcoin, which was designed primarily as a peer-to-peer electronic cash system, Ethereum introduced a Turing-complete virtual machine — the Ethereum Virtual Machine (EVM) — enabling developers to deploy self-executing programmes called smart contracts. These contracts underpin the broader ecosystem of decentralised applications (dApps), decentralised finance (DeFi), and non-fungible tokens (NFTs). As of 2024, Ethereum hosts over $40 billion in total value locked (TVL) across its DeFi protocols and processes millions of transactions daily.

Every Ethereum transaction is permanently recorded on a public ledger, creating a transparent and immutable audit trail. Accounts (wallet addresses) interact by sending Ether (ETH) — the native currency — and ERC-20 tokens, which are fungible tokens conforming to a standardised smart contract interface. This rich, publicly accessible transaction history forms the foundation of the feature engineering performed in this project.

2.2 The Problem of Blockchain Fraud

Despite its transparency, the pseudonymous nature of Ethereum addresses creates significant opportunities for illicit activity. Common forms of Ethereum-based fraud include Ponzi schemes, rug pulls, phishing scams, money laundering, and exchange hacks. In a rug pull, developers of a token project artificially inflate token value before withdrawing all liquidity and abandoning investors. Phishing attackers deploy counterfeit contract addresses or front-end interfaces to steal funds. According to Chainalysis, illicit cryptocurrency addresses received over $24 billion in 2023 alone.

Traditional fraud detection methods rely on centralised Know-Your-Customer (KYC) checks and rule-based transaction monitoring. These approaches are fundamentally inadequate in a permissionless, pseudonymous blockchain environment where anyone can generate a new address instantly and where transaction volumes make manual review infeasible.

2.3 Role of Machine Learning

Machine learning offers a powerful alternative: rather than relying on predefined rules, ML models learn discriminative patterns from historical labelled data. On-chain transaction behaviour of fraudulent addresses often exhibits statistically anomalous patterns — unusually rapid fund movement, abnormal transaction frequency distributions, or interaction with a concentrated set of counterpart addresses. These patterns manifest as statistical features computable directly from the public Ethereum ledger, enabling supervised binary classification.

3 Problem Definition

The core task is a supervised binary classification problem formally defined as follows:

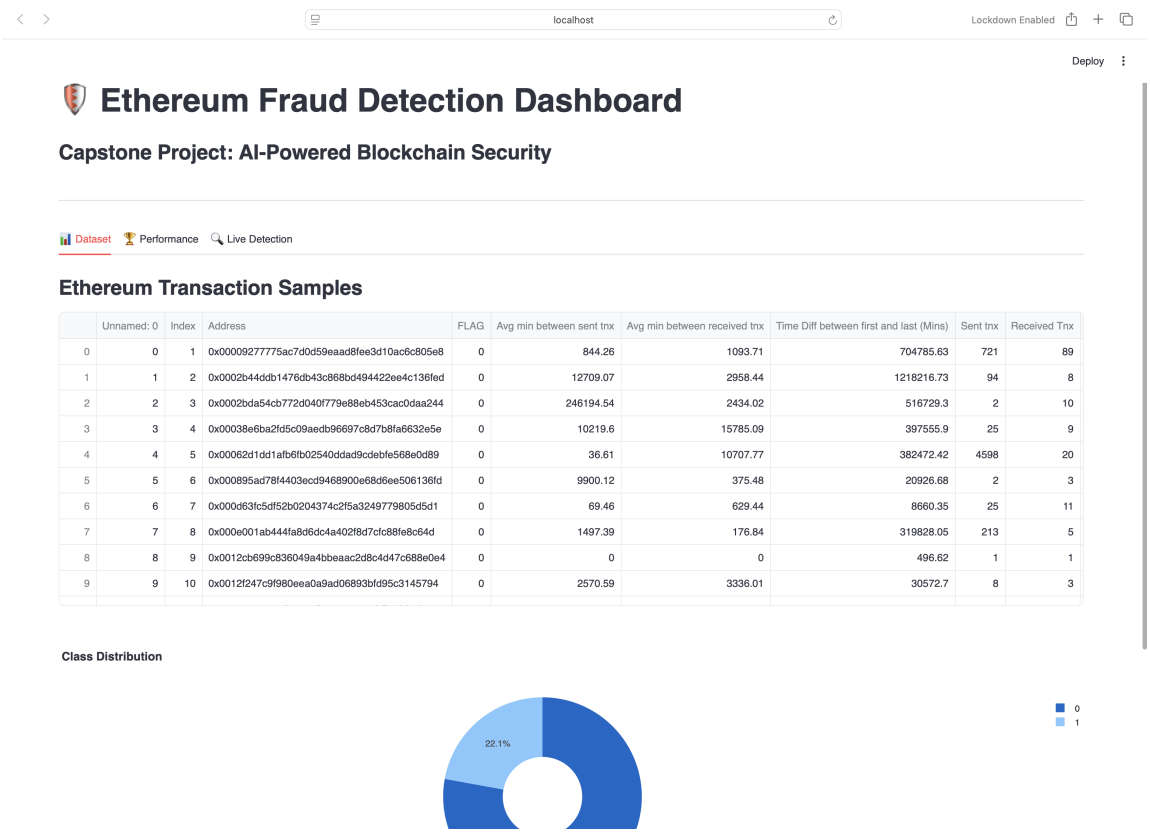
Given a feature vector $x \in \mathbb{R}^d$ representing the aggregated transaction statistics of an Ethereum wallet address, predict the binary label $y \in \{0, 1\}$, where:

- $y = 0$ indicates a legitimate (non-fraudulent) address
- $y = 1$ indicates a fraudulent (illicit) address

The model is trained on a labelled dataset $D = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\}$ where $n = 9,841$ and the feature dimension $d = 47$ (after preprocessing). The task objective is to learn a mapping $f : \mathbb{R}^d \to \{0, 1\}$ that minimises misclassification, with particular emphasis on minimising false negatives (missed fraud cases), as failing to detect a fraudulent address carries substantially higher real-world cost than a false alarm.

4 Dataset Description

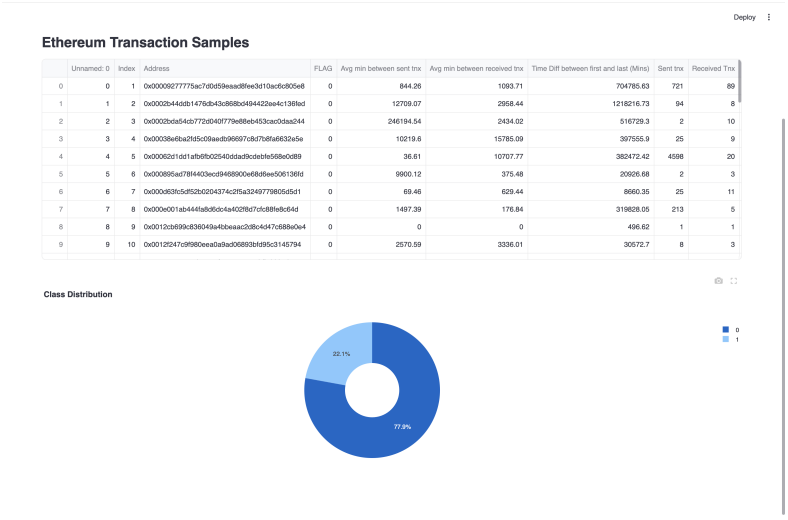
The dataset used in this project is a curated collection of Ethereum account transaction records sourced from the Ethereum blockchain via Etherscan's public API. The dataset contains 9,841 labelled addresses and 51 raw columns.



	Unnamed: 0	Index	Address	FLAG	Avg min between sent tnx	Avg min between received tnx	Time Diff between first and last (Mins)	Sent tnx	Received Tnx
0	0	1	0x00009277775ac7d0d59eaad8fee3d10ac6c805e8	0	844.26	1093.71	704785.63	721	89
1	1	2	0x0002b44ddb1476db43c868bd494422ee4c136fed	0	12709.07	2958.44	1218216.73	94	8
2	2	3	0x0002bda54cb772d040f779e88eb453cac0daa244	0	246194.54	2434.02	516729.3	2	10
3	3	4	0x00038e6ba2fd5c09aedb96697c8d7b8fa6632e5e	0	10219.6	15785.09	397555.9	25	9
4	4	5	0x00062d1dd1afb6fb02540ddad9cdebfe568e0d89	0	36.61	10707.77	382472.42	4598	20
5	5	6	0x000895ad78f4403ecd9468900e68d6ee506136fd	0	9900.12	375.48	20926.68	2	3
6	6	7	0x000d63fc5df52b0204374c2f5a3249779805d5d1	0	69.46	629.44	8660.35	25	11
7	7	8	0x000e001ab444fa8d6dc4a402f8d7cfc88fe8c64d	0	1497.39	176.84	319828.05	213	5
8	8	9	0x0012cb699c836049a4bbeaac2d8c4d47c688e0e4	0	0	0	496.62	1	1
9	9	10	0x0012f247c9f980eea0a9ad06893bfd95c3145794	0	2570.59	3336.01	30572.7	8	3

Figure 1: Interface showing Ethereum transaction samples used for analysis.

The dataset exhibits a moderate class imbalance: approximately 77.9% legitimate addresses versus 22.1% fraudulent addresses. This motivates the use of metrics beyond raw accuracy — specifically, recall on the fraud class.



Deploy

Ethereum Transaction Samples

	Unnamed: 0	Index	Address	FLAG	Avg min between sent tnx	Avg min between received tnx	Time Diff between first and last (Mins)	Sent tnx	Received Tnx
0	0	1	0x0000927777 5ac7d0d59eaad8fee3d10ac6c805e8	0	844.26	1093.71	704785.63	721	89
1	1	2	0x0002b44ddb1476db43c868bd494422ea4c136fed	0	12709.07	2958.44	1218216.73	94	8
2	2	3	0x0002bda54cb772d040f779e88eb453cac0daa244	0	246194.54	2434.02	516729.3	2	10
3	3	4	0x00038e6ba2fd5c09aedb96697c8d7b8fa6632e5e	0	10219.6	15785.09	397555.9	25	9
4	4	5	0x00062d1dd1afb6fb02540ddad9cdebfe568e0d89	0	36.61	10707.77	382472.42	4598	20
5	5	6	0x000895ad78f4403ecd9468900e68d6ee506136fd	0	9900.12	375.48	20926.68	2	3
6	6	7	0x000d63fc5df52b0204374c2f5a324977980 5d5d1	0	69.46	629.44	8660.35	25	11
7	7	8	0x000e001ab444fa8d6dc4a402f8d7cfc88fe8c64d	0	1497.39	176.84	319828.05	213	5
8	8	9	0x0012cb699c836049a4bbeaac2c8c4d47c688e0e4	0	0	0	496.62	1	1
9	9	10	0x0012f247c9f980eea0a9ad06893bfd95c3145794	0	2570.59	3336.01	30572.7	8	3

Class Distribution

Figure 2: Class distribution of fraudulent vs legitimate accounts in the dataset.

4.1 Feature Categories

The 47 features fall into two major categories:

1. **Native ETH Transaction Features:** These features summarise the behaviour of the account with respect to standard ETH transfers (e.g., Average min between sent/received tnx, total Ether sent/received, total ether balance).

2. **ERC-20 Token Transaction Features:** ERC-20 features capture behaviour in the token economy, which is where many sophisticated fraud schemes operate (e.g., Total ERC20 tnxs, ERC20 total Ether received/sent, ERC20 most sent token type).

5 Preprocessing Pipeline

The preprocessing steps applied in the system are as follows:

- **Column normalisation:** Spaces replaced with underscores to prevent pandas parsing errors.

- **Dropping administrative columns:** `Unnamed: 0`, `Index`, and `Address` are removed as they carry no predictive signal.

- **Missing value imputation:** All NaN entries are filled with 0, treating missing ERC-20 activity as absent.

- **Categorical encoding:** The two text columns (ERC20 most sent and received token type) are converted to binary indicator columns via pandas `get_dummies()`.
- **Class Balancing (SMOTE):** Applied Synthetic Minority Over-sampling Technique to synthetically balance the Safe and Fraud classes in the training set.
- **Standard scaling:** All features are transformed using a fitted `StandardScaler`, centring each feature to zero mean and unit variance. The scaler is serialised to `scaler.pkl` and reused at inference time.

6 Machine Learning Models

This section provides an in-depth treatment of each model architecture employed in the project, covering theoretical foundations, implementation details, and results.

6.1 XGBoost — eXtreme Gradient Boosting

XGBoost is a scalable, regularised implementation of gradient boosted decision trees. Gradient boosting constructs an ensemble of weak learners sequentially. Formally, the model predicts $\hat{y}_i = \sum_k f_k(x_i)$, where f_k is the k-th regression tree. The objective function minimised during training is:

$$\text{Obj}(\theta) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k) \tag{1}$$

where l is the loss function (binary cross-entropy for classification), and $\Omega(f) = \gamma T + \frac{1}{2}\lambda||w||^2$ is a regularisation term penalising tree complexity. XGBoost approximates the optimal tree at each step using a second-order Taylor expansion of the loss.

6.2 LightGBM — Light Gradient Boosting Machine

LightGBM introduces two key algorithmic innovations over standard GBDT: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). Unlike XGBoost's level-wise (breadth-first) tree growth, LightGBM grows trees leaf-wise (best-first), selecting the leaf with maximum loss reduction at each step. This produces deeper, more asymmetric trees that converge faster but may overfit without depth constraints.

6.3 Random Forest

Random Forest is a bagging ensemble of decision trees. During training, B trees are grown independently on bootstrap samples. The variance reduction achieved by averaging over many decorrelated trees is described by:

$$\text{Var}(\bar{y}) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B} \tag{2}$$

where ρ is the pairwise correlation between trees and σ^2 is the variance of a single tree. Feature subsampling reduces ρ, making the ensemble variance approach 0 as $B \to \infty$.

6.4 Logistic Regression

Logistic Regression is a linear discriminative classifier that models the posterior probability of class membership using the sigmoid function:

$$P(y = 1|x) = \sigma(w^T x + b) = \frac{1}{1 + \exp(-(w^T x + b))} \tag{3}$$

The model parameters $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are estimated by minimising the binary cross-entropy loss with an L2 regularisation penalty:

$$\mathcal{L}(w, b) = -\sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2}||w||^2 \tag{4}$$

The decision boundary is a hyperplane $w^T x + b = 0$ in the feature space.

6.5 K-Nearest Neighbours (KNN)

K-Nearest Neighbours is a non-parametric, instance-based learning algorithm that makes predictions by finding the K training examples closest to the query point:

$$\hat{y} = \text{mode}(\{y_i : x_i \in N_K(x)\}) \tag{5}$$

where $N_K(x)$ denotes the K nearest neighbours of x under a Euclidean distance metric. KNN is well-suited when fraudulent addresses cluster together in feature space due to shared behavioural patterns.

6.6 TensorFlow Deep Neural Network (DNN)

A Deep Neural Network is a multi-layer feedforward network composed of densely connected neurons with non-linear activation functions:

$$h^{(l)} = \sigma(W^{(l)} h^{(l-1)} + b^{(l)}) \tag{6}$$

$$\hat{y} = \text{sigmoid}(W^{(L+1)} h^{(L)} + b^{(L+1)}) \tag{7}$$

The network is trained end-to-end by backpropagating gradients using the Adam optimiser. Techniques such as batch normalisation and dropout regularisation are employed. A fraud decision threshold of 0.4 (rather than the default 0.5) is applied to increase sensitivity.

7 Safety Ensemble Strategy

The Safety Ensemble aggregates predictions from all six individual models (XGBoost, LightGBM, Random Forest, Logistic Regression, KNN, and TF DNN) using a majority-vote rule. An address is classified as fraudulent if at least 2 of the 6 models predict fraud:

$$\text{ensemble_pred} = 1 \text{ if } \sum_i \text{pred}_i \geq 2 \text{ else } 0 \tag{8}$$

This low threshold (2 out of 6) is deliberately calibrated to maximise fraud recall at the cost of a small increase in false positives — a trade-off entirely appropriate in security-critical fraud detection contexts where the consequence of a missed fraud is far greater than that of a false alarm.

8 Model Comparison and Analysis

Model	**Accuracy**	**Recall (Fraud)**	**Complexity**
XGBoost	99.49%	97.71%	High
LightGBM	99.44%	97.71%	High
Random Forest	99.09%	95.87%	Medium-High
Logistic Regression	97.77%	96.56%	Simple
KNN	96.65%	95.18%	Medium
TF DNN	∼98.5%	∼96.0%	High
Safety Ensemble	**99.58%**	**98.50%**	**Composite**

Table 1: Model Performance Comparison Summary.

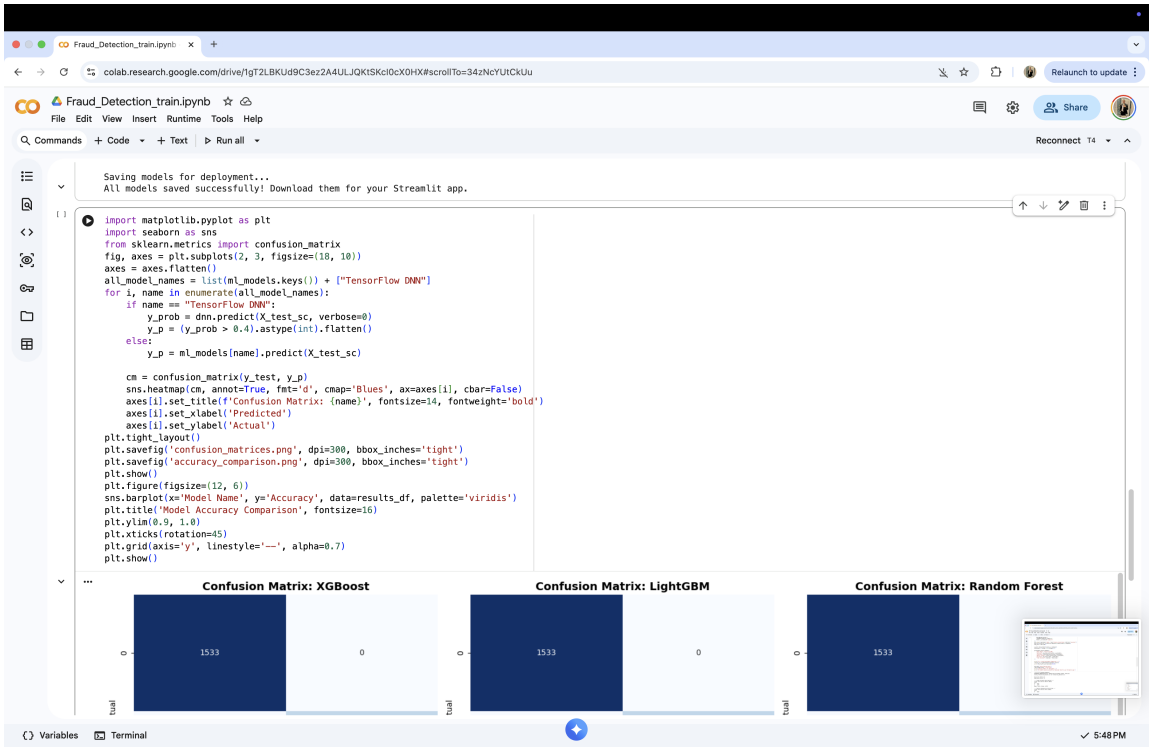


Figure 3: Model performance visualization from the training environment.

Gradient boosting methods (XGBoost and LightGBM) outperform all other single models in accuracy. Both achieve identical fraud recall (97.71%), suggesting they discover similar discriminative patterns. The Safety Ensemble achieves the highest performance on both metrics (99.58% accuracy, 98.50% recall), demonstrating the value of aggregating diverse model architectures.

9 Deployment: Streamlit Application

The project is deployed as a Streamlit web application (`app.py`) providing a browser-accessible fraud detection interface. The application is structured around three functional tabs: Dataset, Performance, and Live Detection.

Model and scaler loading is wrapped in a `@st.cache_resource` decorator, ensuring that the six model files and the scaler are loaded from disk only once per server session. The `preprocess_input()` function replicates the exact training-time preprocessing pipeline to prevent any train-test distribution mismatch at inference.

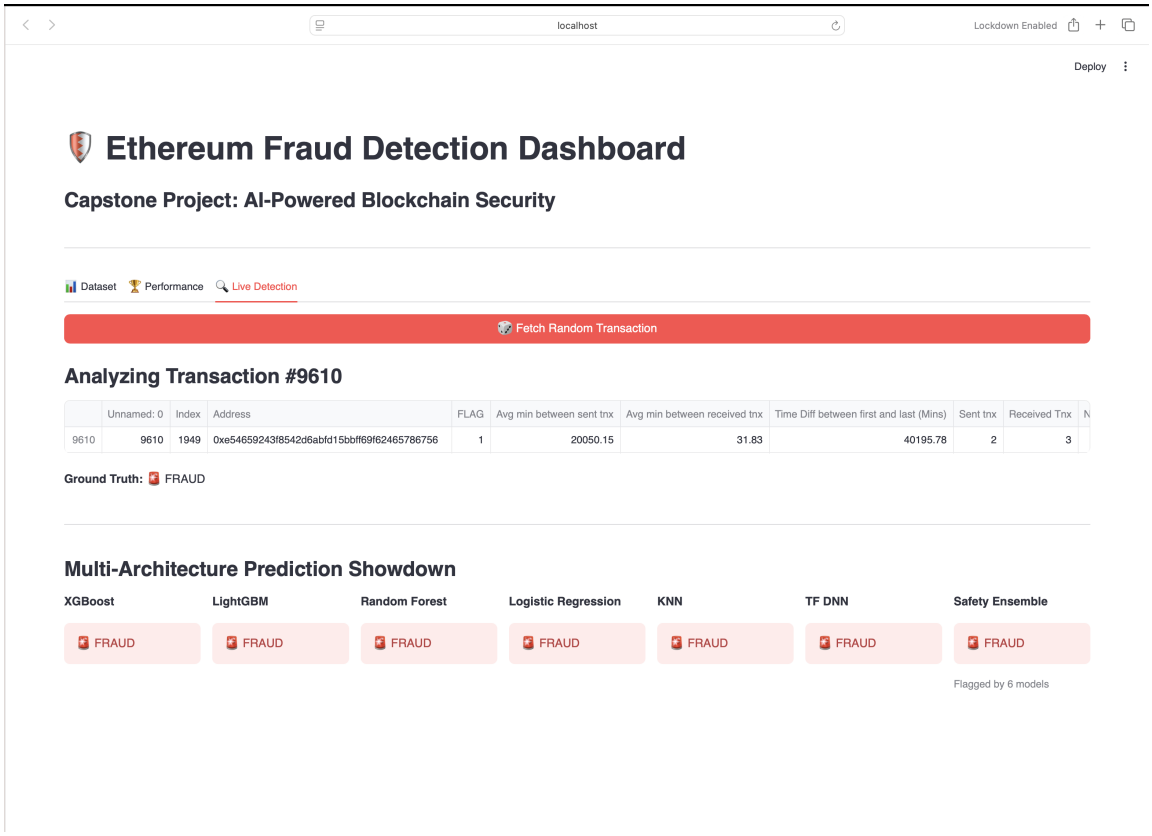


Figure 4: Live detection showing the Safety Ensemble flagging a fraudulent transaction.

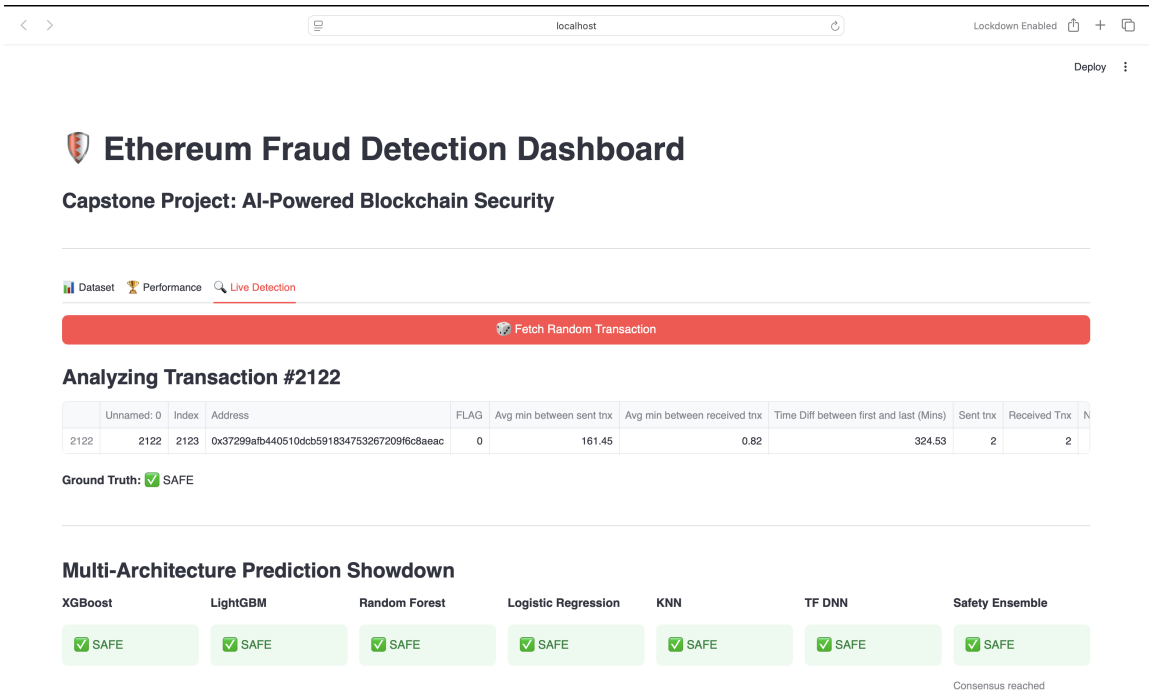


Figure 5: Multi-model consensus identifying a safe transaction.

10 Conclusion

This project has demonstrated that machine learning can achieve near-perfect fraud detection accuracy on Ethereum blockchain transaction data. By engineering 47 behavioural and statistical features from on-chain Ethereum and ERC-20 transaction records, training six diverse ML architectures, and combining them into a Safety Ensemble, the system achieves 99.58% accuracy and 98.50% fraud recall — metrics that are both practically significant and academically noteworthy.

The full system is packaged as an accessible web application, demonstrating a path from research artefact to deployable product. The publicly auditable transaction history provides a rich, label-free feature space that can be exploited to protect users and the wider DeFi ecosystem.

References

[1] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.

[2] Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Cox, D. R. (1958). The Regression Analysis of Binary Sequences. JRSS-B, 20(2), 215–242.

[5] Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory, 13(1), 21–27.

[6] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. Nature, 521(7553), 436–444.

[7] Chainalysis. (2024). The Chainalysis 2024 Crypto Crime Report.

[8] Ethereum Foundation. (2024). Ethereum Documentation. https://ethereum.org

[9] Kaggle. (2020). Ethereum Fraud Detection Dataset.

[10] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

11 Data and Code Availability

The dataset used for training and evaluating the models in this project is publicly available on Kaggle. The data consists of Ethereum transaction logs, engineered features, and ERC-20 token transfer statistics.

- **Dataset Source:** The raw and preprocessed dataset can be accessed via Kaggle [9].
- **Project Codebase:** The complete source code, including the Jupyter notebooks for model training (`Fraud_Detection_train.ipynb`), the serialized machine learning models (`.pkl` and `.keras`), and the Streamlit web application deployment code (`app.py`), have been compiled and packaged as part of the project submission.
- **GitHub Repository:** The project is open-source and the full repository can be accessed at `https://github.com/KareemA4raf/Crypto-Fraud-Detection`.